

LAPORAN PRAKTIKUM PEKAN 5

STRUKTUR DATA

Disusun Oleh:

Syasya Halwa Gazwani

(2511531018)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid Alvero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG
2026

KATA PENGANTAR

Assalamu'alaikum warahmatullahi wabarakatuh,

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW. yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini. Penulis mengucapkan terima kasih kepada bapak Dr. Wahyudi, S.T, M.T. selaku dosen mata kuliah Struktur Data yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan mata kuliah yang penulis tekuni. Penulis juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga penulis dapat menyelesaikan Laporan Praktikum ini.

Padang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
DASAR TEORI	2
2.1 NodeDLL_2511531018.....	2
2.1.1 Kode Program	2
2.1.2 Langkah Kerja	3
2.2 insertDLL_2511531018.....	4
2.2.1. Kode Program	4
2.2.2. Langkah Kerja	6
2.2.3 Output	10
2.3 PenelusuranDLL_2511531018	10
2.3.1 Kode Program	10
2.3.2 Langkah Kerja	11
2.4.3 Output	12
2.4 HapusDLL_2511531018.....	13
2.4.1 Kode Program	13
2.4.2 Langkah Kerja	15
2.4.3 Output	18
2.6 Tugas.....	18

2.6.1	Class Lagu_2511531018.....	18
2.6.2	Class Musik_2511531018.....	19
2.6.3	Output	23
BAB III PENUTUP		26
DAFTAR PUSTAKA.....		27

BAB I PENDAHULUAN

1.1 Latar Belakang

Double Linked List adalah salah satu jenis struktur data linked list yang terdiri dari sekumpulan node yang saling terhubung satu sama lain. Setiap node menyimpan data serta memiliki dua pointer, yaitu next dan prev. Pointer next digunakan untuk menunjuk ke node berikutnya, sedangkan pointer prev digunakan untuk menunjuk ke node sebelumnya. Dengan adanya dua pointer tersebut, hubungan antar node menjadi dua arah.

Karena dapat diakses dari dua arah, proses penelusuran data pada Double Linked List menjadi lebih fleksibel. Data tidak hanya dapat ditelusuri dari awal ke akhir, tetapi juga dapat ditelusuri kembali dari akhir ke awal. Hal ini berbeda dengan Single Linked List yang hanya memiliki satu pointer next, sehingga penelusuran data hanya bisa dilakukan satu arah saja. Pada Double Linked List, perpindahan antar node menjadi lebih mudah karena setiap node mengetahui node sebelum dan sesudahnya.

1.2 Tujuan

- 1.2.1 Meningkatkan pemahaman tentang Double Linked List.
- 1.2.2 Dapat membuat sebuah program dengan menggunakan struktur data Double Linked List.
- 1.2.3 Melatih kemampuan logika pemrograman.

1.3 Manfaat Praktikum

- 1.3.1 Menambah pemahaman tentang struktur data Double Linked List.
- 1.3.2 Melatih kemampuan membuat program menggunakan bahasa Java.
- 1.3.3 Memahami proses penambahan, penghapusan, pencarian, dan penelusuran data dua arah.

BAB II PEMBAHASAN

DASAR TEORI

Double Linked List (DLL) salah satu jenis struktur data linked list yang terdiri dari sekumpulan node yang saling terhubung satu sama lain. Setiap node menyimpan data serta memiliki dua pointer, yaitu next dan prev. Pointer next digunakan untuk menunjuk ke node berikutnya, sedangkan pointer prev digunakan untuk menunjuk ke node sebelumnya. Dengan adanya dua pointer tersebut, hubungan antar node menjadi dua arah.

Karena dapat diakses dari dua arah, proses penelusuran data pada Double Linked List menjadi lebih fleksibel. Data tidak hanya dapat ditelusuri dari awal ke akhir, tetapi juga dapat ditelusuri kembali dari akhir ke awal. Hal ini berbeda dengan Single Linked List yang hanya memiliki satu pointer next, sehingga penelusuran data hanya bisa dilakukan satu arah saja. Pada Double Linked List, perpindahan antar node menjadi lebih mudah karena setiap node mengetahui node sebelum dan sesudahnya.

2.1 NodeDLL_2511531018

2.1.1 Kode Program

```
package pekan6;

public class NodeDLL_2511531018 {
    // mendefinisikan kelas node
    int data_1018; // data
    NodeDLL_2511531018 next_1018; // Pointer ke next node
    NodeDLL_2511531018 prev_1018; // Pointer ke previous node

    // Konstruktor
    public NodeDLL_2511531018(int data_1018) {
        this.data_1018 = data_1018;
        this.next_1018 = null;
        this.prev_1018 = null;
    }
}
```

Gambar 2.1.1 Kode program NodeDLL_2511531018.

2.1.2 Langkah Kerja

- 2.1.2.1 Variabel `data_1018` digunakan untuk menyimpan nilai pada node. Tipe datanya `int`, sehingga hanya dapat menyimpan angka bilangan bulat.

```
// mendefinisikan kelas node
int data_1018; // data
```

Gambar 2.1.2.1 Variabel.

- 2.1.2.2 Variabel `next_1018` untuk menyimpan alamat node berikutnya, digunakan untuk bergerak maju pada linked list. Variabel `prev_1018` untuk menyimpan alamat node sebelumnya, digunakan untuk bergerak mundur pada linked list.

```
NodeDLL_2511531018 next_1018; // Pointer ke next node
NodeDLL_2511531018 prev_1018; // Pointer ke previous
node
```

Gambar 2.1.2.2 Fungsi Variabel .

- 2.1.2.3 Konstruktor dipanggil otomatis saat objek dibuat. Parameter `data_1018` digunakan untuk mengisi data node.

```
// Konstruktor
public NodeDLL_2511531018(int data_1018) {
```

Gambar 2.1.2.3 Konstruktor class.

- 2.1.2.4 `This.data_1018` merupakan variabel milik object sedangkan `data_1018` merupakan parameter dari konstruktor. Lalu, awalnya node belum terhubung ke node berikutnya, maka `next_1018` diisi `null`. Awalnya node juga belum punya node sebelumnya. Maka `prev_1018` diisi `null`.

```
this.data_1018 = data_1018;
this.next_1018 = null;
this.prev_1018 = null;
```

Gambar 2.1.2.4 Menyimpan data, mengambil node setelahnya, dan mengambil node sebelumnya.

2.2 insertDLL_2511531018

2.2.1. Kode Program

```
package pekan6;

public class insertDLL_2511531018 {
    // menambahkan node diawal DLL
    static NodeDLL_2511531018
insertBegin_1018(NodeDLL_2511531018 head_1018, int data_1018)
{
    // buat node baru
    NodeDLL_2511531018 new_node_1018 = new
NodeDLL_2511531018(data_1018);
    // jadikan pointer nextnya head
    new_node_1018.next_1018 = head_1018;
    // jadikan pointer prev head ke new_node
    if (head_1018 != null) {
        head_1018.prev_1018 = new_node_1018;
    }
    return new_node_1018;
}
    //fungsi menambahkan node di akhir
    public static NodeDLL_2511531018
insertEnd_1018(NodeDLL_2511531018 head_1018, int
newData_1018) {
    //buat node baru
    NodeDLL_2511531018 newNode_1018 = new
NodeDLL_2511531018(newData_1018);
    // jika dll null jadikan head
    if (head_1018 == null) {
        head_1018 = newNode_1018;
    }
    else {
        NodeDLL_2511531018 curr_1018 = head_1018;
        while (curr_1018.next_1018 != null) {
            curr_1018 = curr_1018.next_1018;
        }
        curr_1018.next_1018 = newNode_1018;
        newNode_1018.prev_1018 = curr_1018;
    }
    return head_1018;
}
    // fungsi menambahkan node di posisi tertentu
    public static NodeDLL_2511531018
insertAtPosition_1018(NodeDLL_2511531018 head_1018, int
pos_1018, int new_data_1018) {
    // buat node baru
    NodeDLL_2511531018 new_node_1018 = new
NodeDLL_2511531018(new_data_1018);
    if (pos_1018 == 1) {
        new_node_1018.next_1018 = head_1018;
        if (head_1018 != null) {
            head_1018.prev_1018 = new_node_1018; }
        head_1018 = new_node_1018;
        return head_1018; }
}
```



```

        NodeDLL_2511531018 curr_1018 = head_1018;
        for(int i_1018 = 1; i_1018 < pos_1018 - 1 && curr_1018 !=
null; ++i_1018) {
            curr_1018 = curr_1018.next_1018; }
        if (curr_1018 == null) {
            System.out.println("Posisi tidak ada");
            return head_1018; }
        new_node_1018.prev_1018 = curr_1018;
        new_node_1018.next_1018 = curr_1018.next_1018;
        curr_1018.next_1018 = new_node_1018;
        if (new_node_1018.next_1018 != null) {
            new_node_1018.next_1018.prev_1018 =
new_node_1018; }
        return head_1018;
    }

    public static void printList_1018(NodeDLL_2511531018
head_1018) {
        NodeDLL_2511531018 curr_1018 = head_1018;
        while (curr_1018 != null) {
            System.out.print(curr_1018.data_1018 +
" <-> ");
            curr_1018 = curr_1018.next_1018;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // membuat dll 2 <-> 3 <-> 5
        NodeDLL_2511531018 head_1018 = new
NodeDLL_2511531018(2);
        head_1018.next_1018 = new
NodeDLL_2511531018(3);
        head_1018.next_1018.prev_1018 = head_1018;
        head_1018.next_1018.next_1018 = new
NodeDLL_2511531018(5);
        head_1018.next_1018.next_1018.prev_1018 =
head_1018.next_1018;
        //cetak DLLawal
        System.out.print("DLL Awal: ");
        printList_1018(head_1018);
        //tambah 1 di awal
        head_1018 = insertBegin_1018(head_1018, 1);
        System.out.print(
            "simpul 1 ditambah di awal: ");
        printList_1018(head_1018);
        // tambah 6 di akhir
        System.out.print(
            "simpul 6 ditambah di
akhir: ");
        int data_1018 = 6;
        head_1018 = insertEnd_1018(head_1018,
data_1018);
        printList_1018(head_1018);
        //menambah node 4 di posisi 4
        System.out.print("tambah node 4 di
posisi 4: ");
    }

```

```

        int data2_1018 = 4;
        int pos_1018 = 4;
        head_1018 =
insertAtPosition_1018(head_1018, pos_1018, data2_1018);
        printList_1018(head_1018);
    }
}

```

Gambar 2.2.1 Kode program insertDLL_2511531018.

2.2.2. Langkah Kerja

2.2.2.1 Method untuk menambahkan node diawal DLL.

```

static NodeDLL_2511531018
insertBegin_1018(NodeDLL_2511531018 head_1018, int
data_1018) {

```

Gambar 2.2.2.1 Method insert awal.

2.2.2.2 Membuat node baru dengan data dari parameter.

```

NodeDLL_2511531018 new_node_1018 = new
NodeDLL_2511531018(data_1018);

```

Gambar 2.2.2.2 Node baru.

2.2.2.3 Menghubungkan node baru ke head lama. Jika list tidak kosong, maka prev head lama diarahkan ke node baru. Lalu, mengembalikan head baru.

```

new_node_1018.next_1018 = head_1018;
// jadikan pointer prev head ke new_node
if (head_1018 != null) {
    head_1018.prev_1018 = new_node_1018;
}
return new_node_1018;

```

Gambar 2.2.2.3 Method insert di awal.

2.2.2.4 Method untuk menambahkan node di akhir linked list. Membuat node baru.

```

public static NodeDLL_2511531018
insertEnd_1018(NodeDLL_2511531018 head_1018, int
newData_1018) {
    //buat node baru
    NodeDLL_2511531018 newNode_1018 = new
NodeDLL_2511531018(newData_1018);

```

Gambar 2.2.2.4 Method insert di akhir.

2.2.2.5 Jika belum ada node, maka node baru dimasukkan akan menjadi head.

```

if (head_1018 == null) {
    head_1018 = newNode_1018;
}

```

Gambar 2.2.2.5 Method insert di akhir.

- 2.2.2.6 Traversal ke node terakhir yaitu bergerak sampai node terakhir, lalu menghubungkan node terakhir dengan node baru.

```
NodeDLL_2511531018 curr_1018 = head_1018;
    while (curr_1018.next_1018 != null) {
        curr_1018 = curr_1018.next_1018;
    }
    curr_1018.next_1018 = newNode_1018;
    newNode_1018.prev_1018 = curr_1018;
```

Gambar 2.2.2.6 Method insert di akhir.

- 2.2.2.7 Menambahkan node pada posisi tertentu dengan membuat node baru.

```
public static NodeDLL_2511531018
insertAtPosition_1018(NodeDLL_2511531018 head_1018,
int pos_1018, int new_data_1018) {
NodeDLL_2511531018 new_node_1018 = new
NodeDLL_2511531018(new_data_1018);
```

Gambar 2.2.2.7 Method insert di posisi tertentu.

- 2.2.2.8 Jika posisi yang diminta adalah posisi pertama, maka node baru harus ditambahkan diawal linked list. Pointer next dari node baru diarahkan ke head lama, lalu dicek apakah linked list kosong atau tidak, jika list kosong, maka tidak perlu mengatur pointer prev. Setelah itu, mengubah pointer prev milik head lama agar menunjuk ke node baru.

```
if (pos_1018 == 1) {
    new_node_1018.next_1018 = head_1018;
    if (head_1018 != null) {
        head_1018.prev_1018 =
new_node_1018; }
    head_1018 = new_node_1018;
    return head_1018; }
```

Gambar 2.2.2.8 Method insert di posisi tertentu.

- 2.2.2.9 Melakukan perpindahan node sampai ke posisi sebelum target insert.

```
for(int i_1018 = 1; i_1018 < pos_1018 - 1 && curr_1018
!= null; ++i_1018) {
    curr_1018 = curr_1018.next_1018; }
```

Gambar 2.2.2.9 Method insert di posisi tertentu.

- 2.2.2.10 Jika traversal sudah sampai akhir linked list, posisi tersebut tidak tersedia.

```
if (curr_1018 == null) {
    System.out.println("Posisi tidak ada");
    return head_1018; }
```

Gambar 2.2.2.10 Method insert di posisi tertentu.

- 2.2.2.11 Menghubungkan prev node baru, menghubungkan next node baru, menghubungkan node sebelumnya ke node baru, lalu mengecek apakah ada node setelahnya yaitu digunakan untuk memastikan node baru bukan node terakhir, jika setelah node baru masih ada node yang lain, maka pointer prev node tersebut harus diperbaiki, setelah itu menghubungkan prev node setelahnya..

```
new_node_1018.prev_1018 = curr_1018;
new_node_1018.next_1018 =
curr_1018.next_1018;
curr_1018.next_1018 = new_node_1018;
if (new_node_1018.next_1018 != null) {
    new_node_1018.next_1018.prev_1018 =
new_node_1018; }
return head_1018;
```

Gambar 2.2.2.11 Method insert di posisi tertentu.

- 2.2.2.12 Digunakan untuk mencetak seluruh isi node dari awal sampai akhir. Curr_1018 dipakai untuk berjalan menyusuri linked list yang awalnya menunjuk ke head. Loop akan tetap terus berjalan selama belum mencapai akhir linked list. Lalu menampilkan data node satu persatu dan pindah ke node berikutnya.

```
public static void printList_1018(NodeDLL_2511531018
head_1018) {
    NodeDLL_2511531018 curr_1018 =
head_1018;
    while (curr_1018 != null) {
        System.out.print(curr_1018.data_1018 + " <-> ");
        curr_1018 = curr_1018.next_1018;
    }
```

Gambar 2.2.2.12 Method printList.

- 2.2.2.13 Program akan dijalankan dengan membuat node pertama terlebih dahulu, lalu menambahkan node 3 dan menghubungkan prev node, lalu menambahkan node 5 dan menghubungkan prev node 5.

```

public static void main(String[] args) {
    // membuat dll 2 <-> 3 <-> 5
    NodeDLL_2511531018 head_1018 = new
NodeDLL_2511531018(2);
    head_1018.next_1018 = new
NodeDLL_2511531018(3);
    head_1018.next_1018.prev_1018 =
head_1018;
    head_1018.next_1018.next_1018 = new
NodeDLL_2511531018(5);
    head_1018.next_1018.next_1018.prev_1018
= head_1018.next_1018;
}

```

Gambar 2.2.2.13 Method main.

2.2.2.14 Mencetak DLL awal dan menambahkan node 1 di awal, lalu mencetak hasilnya.

```

//cetak DLLawal
System.out.print("DLL Awal: ");
printList_1018(head_1018);
//tambah 1 di awal
head_1018 = insertBegin_1018(head_1018,
1);
System.out.print(
    "simpul 1 ditambah di
awal: ");
printList_1018(head_1018);
// tambah 6 di akhir
System.out.print(
    "simpul 6 ditambah
di ahir: ");
}

```

Gambar 2.2.2.14 Method main.

2.2.2.15 Menambahkan node 6 di akhir, lalu mencetak hasilnya. Menambahkan node 4 pada posisi 4 dan memanggil method insert posisi dan mencetak hasil akhirnya.

```

int data_1018 = 6;
head_1018 =
insertEnd_1018(head_1018, data_1018);
printList_1018(head_1018);
//menambah node 4 di posisi 4
System.out.print("tambah node 4
di posisi 4: ");
int data2_1018 = 4;
int pos_1018 = 4;
head_1018 =
insertAtPosition_1018(head_1018, pos_1018,
data2_1018);
printList_1018(head_1018);
}

```

Gambar 2.2.2.15 Method main.

2.2.3 Output

```
DLL Awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di ahir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

Gambar 2.2.3 Output insertDLL_2511531018.

2.3 PenelusuranDLL_2511531018

2.3.1 Kode Program

```
package pekan6;

public class PenelusuranDLL_2511531018 {
    // fungsi penelusuran maju
    static void forwardTraversal_1018(NodeDLL_2511531018
head_1018) {
        // memulai penelusuran dari head
        NodeDLL_2511531018 curr_1018 = head_1018;
        // lanjutkan sampai akhir
        while (curr_1018 != null) {
            //print data
            System.out.print(curr_1018.data_1018 + " <->
");
            // pindah ke node berikutnya
            curr_1018 = curr_1018.next_1018;
        }
        //print spasi
        System.out.println();
    }
    // fungsi penelusuran mun dur
    static void backwardTraversal_1018(NodeDLL_2511531018
tail_1018) {
        // mulai dari akhir
        NodeDLL_2511531018 curr_1018 = tail_1018;
        //lanjut sampai head
        while (curr_1018 != null) {
            // cetak data
            System.out.print(curr_1018.data_1018 + " <->
");
            //pindah ke node sebelumnya
            curr_1018 = curr_1018.prev_1018;
        }
        //cetak spasi
        System.out.println();
    }
    public static void main(String[] args) {
        // cetak DLL
        NodeDLL_2511531018 head_1018 = new
NodeDLL_2511531018(1);
        NodeDLL_2511531018 second_1018 = new
NodeDLL_2511531018(2);
```

```

        NodeDLL_2511531018 third_1018 = new
NodeDLL_2511531018(3);

        head_1018.next_1018 = second_1018;
        second_1018.prev_1018 = head_1018;
        second_1018.next_1018 = third_1018;
        third_1018.prev_1018 = second_1018;

        System.out.println("Penelusuran maju:");
        forwardTraversal_1018(head_1018);

        System.out.println("Penelusuran mundur:");
        backwardTraversal_1018(third_1018);
    }
}

```

Gambar 2.3.1 Kode program PenelusuranDLL_2511531018.

2.3.2 Langkah Kerja

2.3.2.1 Method ini digunakan untuk menelusuri DLL dari depan ke belakang. Program berjalan menelusuri node, loop berjalan selama node masih ada dan berhenti saat mencapai null. Lalu menampilkan data node dan pindah ke node berikutnya.

```

static void forwardTraversal_1018(NodeDLL_2511531018
head_1018) {
    // memulai penelusuran dari head
    NodeDLL_2511531018 curr_1018 = head_1018;
    // lanjutkan sampai akhir
    while (curr_1018 != null) {
        //print data
        System.out.print(curr_1018.data_1018 +
" <-> ");
        // pindah ke node berikutnya
        curr_1018 = curr_1018.next_1018;
    }
}

```

Gambar 2.3.2.1 Method forwardTraversal.

2.3.2.2 Method ini digunakan untuk menelusuri DLL dari belakang ke depan. Memulai raversal dari tail, loop traversal berjalan selama node masih ada. Lalu menampilkan data dan bergerak ke node sebelumnya.

```

static void backwardTraversal_1018(NodeDLL_2511531018
tail_1018) {
    // mulai dari akhir
    NodeDLL_2511531018 curr_1018 = tail_1018;
    //lanjut sampai head
    while (curr_1018 != null) {
        // cetak data
    }
}

```

```

        System.out.print(curr_1018.data_1018 +
" <-> ");
        //pindah ke node sebelumnya
        curr_1018 = curr_1018.prev_1018;

```

Gambar 2.3.2.2 Method backwardTraversal.

2.3.2.3 Membuat node pertama, kedua, dan ketiga.

```

public static void main(String[] args) {
    // cetak DLL
    NodeDLL_2511531018 head_1018 = new
NodeDLL_2511531018(1);
    NodeDLL_2511531018 second_1018 = new
NodeDLL_2511531018(2);
    NodeDLL_2511531018 third_1018 = new
NodeDLL_2511531018(3);

```

Gambar 2.3.2.3 Method main.

2.3.2.4 Menghubungkan node pertama dan kedua, lalu menghubungkan prev node kedua ke node pertama, menghubungkan node kedua ke node ketiga, menghubungkan prev node ketiga ke node kedua, lalu menampilkan tulisan dan hasil.

```

head_1018.next_1018 = second_1018;
second_1018.prev_1018 = head_1018;
second_1018.next_1018 = third_1018;
third_1018.prev_1018 = second_1018;

System.out.println("Penelusuran maju:");
forwardTraversal_1018(head_1018);

System.out.println("Penelusuran mundur:");
backwardTraversal_1018(third_1018);

```

Gambar 2.3.2.4 Method main.

2.4.3 Output

```

Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->

```

Gambar 2.3.3 Output PenelusuranDLL_2511531018.

2.4 HapusDLL_2511531018

2.4.1 Kode Program

```
package pekan6;

public class HapusDLL_2511531018 {
    // fungsi menghapus node awal
    public static NodeDLL_2511531018
delHead_1018(NodeDLL_2511531018 head_1018) {
    if (head_1018 == null) {
        return null; }
    NodeDLL_2511531018 temp_1018 = head_1018;
    head_1018 = head_1018.next_1018;
    if (head_1018 != null) {
        head_1018.prev_1018 = null; }
    return head_1018;
    }
    //fungsi menghapus di akhir
    public static NodeDLL_2511531018
delLast_1018(NodeDLL_2511531018 head_1018) {
    if (head_1018 == null) {
        return null; }
    if (head_1018.next_1018 == null) {
        return null; }
    NodeDLL_2511531018 curr_1018 =
head_1018;

    while (curr_1018.next_1018 != null) {
        curr_1018 = curr_1018.next_1018;
    }
    //update pointer previous node
    if (curr_1018.prev_1018 != null) {
        curr_1018.prev_1018.next_1018 =
null; }

    return head_1018;
    }
    // fungsi menghapus node posisi tertentu
    public static NodeDLL_2511531018
delPos_1018(NodeDLL_2511531018 head_1018, int pos_1018) {
    // JIKA DLL kosong
    if (head_1018 == null) {
        return head_1018; }
    NodeDLL_2511531018 curr_1018 =
head_1018;

    //telusuri sampai ke node yang akan
    dihapus
    for (int i_1018 = 1; curr_1018 != null
&& i_1018 < pos_1018; ++i_1018) {
        curr_1018 = curr_1018.next_1018;
    }

    // jika posisi tidak ditemukan
    if (curr_1018 == null) {
        return head_1018; }
    // update pointer
    if (curr_1018.prev_1018 != null) {
        curr_1018.prev_1018.next_1018 =
curr_1018.next_1018;}
```

```

        if (curr_1018.next_1018 != null) {
            curr_1018.next_1018.prev_1018 =
curr_1018.prev_1018; }
        // jika yang dihapus head
        if (head_1018 == curr_1018) {
            head_1018 = curr_1018.next_1018;
        }

        return head_1018;
    }

    // fungsi mencetak DLL
    public static void
printList_1018(NodeDLL_2511531018 head_1018) {
        NodeDLL_2511531018 curr_1018 =
head_1018;

        while (curr_1018 != null) {

            System.out.print(curr_1018.data_1018 + " <-> ");
            curr_1018 = curr_1018.next_1018;

        }
        System.out.println();
    }

    public static void main(String[] args) {
        // buat sebuah DLL
        NodeDLL_2511531018 head_1018 = new
NodeDLL_2511531018(1);
        head_1018.next_1018 = new
NodeDLL_2511531018(2);
        head_1018.next_1018.prev_1018 = head_1018;
        head_1018.next_1018.next_1018 = new
NodeDLL_2511531018(3);
        head_1018.next_1018.next_1018.prev_1018 =
head_1018.next_1018;
        head_1018.next_1018.next_1018.next_1018 = new
NodeDLL_2511531018(4);

        head_1018.next_1018.next_1018.next_1018.prev_1018 =
head_1018.next_1018;

        head_1018.next_1018.next_1018.next_1018.next_1018 = new
NodeDLL_2511531018(5);

        head_1018.next_1018.next_1018.next_1018.next_1018.prev_1018
= head_1018.next_1018.next_1018.next_1018;

        System.out.print("DLL Awal: ");
        printList_1018(head_1018);

        System.out.print("Setelah head dihapus: ");
        head_1018 = delHead_1018(head_1018);
        printList_1018(head_1018);

        System.out.print("Setelah node terakhir
dihapus: ");
        head_1018 = dellast_1018(head_1018);
        printList_1018(head_1018);

        System.out.print("menghapus node ke 2: ");

```

```

        head_1018 = delPos_1018(head_1018, 2);

        printList_1018(head_1018);
    }
}

```

Gambar 2.4.1 Kode program HapusDLL_2511531018.

2.4.2 Langkah Kerja

2.4.2.1 Method delHead_1018() digunakan untuk menghapus node pertama(head). Jika linked list kosong, maka tidak ada yang bisa dihapus dan method mengembalikan null. Menyimpan node head lama ke variabel sementara, lalu memindahkan head ke node berikutnya. Menghapus hubungan prev head baru karena head baru tidak memiliki node sebelumnya.

```

public static NodeDLL_2511531018
delHead_1018(NodeDLL_2511531018 head_1018) {
    if (head_1018 == null) {
        return null; }
    NodeDLL_2511531018 temp_1018 =
head_1018;
    head_1018 = head_1018.next_1018;
    if (head_1018 != null) {
        head_1018.prev_1018 = null; }
    return head_1018;
}

```

Gambar 2.4.2.1 Method delHead_1018.

2.4.2.2 Method ini digunakan untuk menghapus node terakhir. Jika DLL kosong, tidak ada node untuk dihapus. Jika hanya ada satu node, maka linked list menjadi kosong. Lalu traversal ke node terakhir, bergerak ke node berikutnya, dan menghapus hubungan node terakhir.

```

public static NodeDLL_2511531018
dellast_1018(NodeDLL_2511531018 head_1018) {
    if (head_1018 == null) {
        return null; }
    if (head_1018.next_1018 == null)
    {
        return null; }
    NodeDLL_2511531018 curr_1018 =
head_1018;
    while (curr_1018.next_1018 !=
null) {
        curr_1018 =
curr_1018.next_1018;
    }
    //update pointer previous node
}

```

```

        if (curr_1018.prev_1018 != null)
        {
            curr_1018.prev_1018.next_1018 = null; }
        return head_1018;
    }

```

Gambar 2.4.2.2 Method delLast_1018.

- 2.4.2.3 Method ini digunakan untuk menghapus node pada posisi tertentu. Jika DLL kosong, membuat pointer traversal, lalu loop berjalan sampai node posisi yang ingin dihapus ditemukan dan bergerak ke node berikutnya.

```

public static NodeDLL_2511531018
delPos_1018(NodeDLL_2511531018 head_1018, int
pos_1018) {
    // JIKA DLL kosong
    if (head_1018 == null) {
        return head_1018; }
    NodeDLL_2511531018 curr_1018 =
head_1018;
    //telusuri sampai ke node yang
akan dihapus
    for (int i_1018 = 1; curr_1018
!= null && i_1018 < pos_1018; ++i_1018) {
        curr_1018 =
curr_1018.next_1018; }
}

```

Gambar 2.4.2.3 Method delPos_1018.

- 2.4.2.4 Jika posisi melebihi jumlah node, list tidak berubah. Lalu menghubungkan node sebelumnya ke node setelahnya dan menghubungkan prev node setelahnya.

```

if (curr_1018 == null) {
    return head_1018; }
// update pointer
if (curr_1018.prev_1018 != null)
{
    curr_1018.prev_1018.next_1018 =
curr_1018.next_1018;}
    if (curr_1018.next_1018 != null)
    {
        curr_1018.next_1018.prev_1018 = curr_1018.prev_1018;
    }
    // jika yang dihapus head
    if (head_1018 == curr_1018) {
        head_1018 =
curr_1018.next_1018; }
    return head_1018;
}

```

Gambar 2.4.2.4 Method delPos_1018.

- 2.4.2.5 Method untuk mencetak isi DLL dengan membuat pointer traversal dan bergerak ke next node.

```
public static void printList_1018(NodeDLL_2511531018
head_1018) {
    NodeDLL_2511531018 curr_1018 =
head_1018;
    while (curr_1018 != null) {
        System.out.print(curr_1018.data_1018 + " <-> ");
        curr_1018 =
curr_1018.next_1018;
    }
}
```

Gambar 2.4.2.5 Method printList_1018.

- 2.4.2.6 Mencetak DLL awal dengan angka 1, 2, 3, 4, dan 5. Lalu di lanjutkan dengan menghapus head yaitu angka 1. Lalu menghapus node terakhir yaitu 5. Dilanjutkan dengan menghapus node posisi 2 yaitu angka 3.

```
public static void main(String[] args) {
    // buat sebuah DLL
    NodeDLL_2511531018 head_1018 = new
NodeDLL_2511531018(1);
    head_1018.next_1018 = new
NodeDLL_2511531018(2);
    head_1018.next_1018.prev_1018 =
head_1018;
    head_1018.next_1018.next_1018 = new
NodeDLL_2511531018(3);
    head_1018.next_1018.next_1018.prev_1018
= head_1018.next_1018;
    head_1018.next_1018.next_1018.next_1018
= new NodeDLL_2511531018(4);

    head_1018.next_1018.next_1018.next_1018.prev_1018 =
head_1018.next_1018;

    head_1018.next_1018.next_1018.next_1018.next_1018 =
new NodeDLL_2511531018(5);

    head_1018.next_1018.next_1018.next_1018.next_1018.pr
ev_1018 = head_1018.next_1018.next_1018.next_1018;
}
```

Gambar 2.4.2.6 Method main.

- 2.4.2.7 Mencetak seluruh program.

```
System.out.print("DLL Awal: ");
printList_1018(head_1018);

System.out.print("Setelah head dihapus:
");
```

```

        head_1018 = delHead_1018(head_1018);
        printList_1018(head_1018);

        System.out.print("Setelah node terakhir
dihapus: ");
        head_1018 = dellast_1018(head_1018);
        printList_1018(head_1018);

        System.out.print("menghapus node ke 2:
");
        head_1018 = delPos_1018(head_1018, 2);

        printList_1018(head_1018);

```

Gambar 2.4.2.7 Print.

2.4.3 Output

```

DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->
Setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->
Setelah node terakhir dihapus: 2 <-> 3 <-> 4 <->
menghapus node ke 2: 2 <-> 4 <->

```

Gambar 2.4.3 Output HapusDLL_2511531018.

2.6 Tugas

2.6.1 Class Lagu_2511531018

```

package pekan6;

public class Lagu_2511531018 {

    // atribut lagu
    String judul_1018;
    String penyanyi_1018;

    // pointer next dan prev
    Lagu_2511531018 next_1018;
    Lagu_2511531018 prev_1018;

    // konstruktor
    public Lagu_2511531018(String judul_1018, String
penyanyi_1018) {
        this.judul_1018 = judul_1018;
        this.penyanyi_1018 = penyanyi_1018;
        this.next_1018 = null;
        this.prev_1018 = null;
    }

    // getter judul
    public String getJudul_1018() {
        return judul_1018;
    }
}

```

```

// setter judul
public void setJudul_1018(String judul_1018) {
    this.judul_1018 = judul_1018;
}

// getter penyanyi
public String getPenyanyi_1018() {
    return penyanyi_1018;
}

// setter penyanyi
public void setPenyanyi_1018(String penyanyi_1018) {
    this.penyanyi_1018 = penyanyi_1018;
}
}

```

Gambar 2.6.1 Kode Program Lagu_2511531018.

2.6.2 Class Musik_2511531018

```

package pekan6;

import java.util.Scanner;

public class Musik_2511531018 {
    // head dan tail playlist
    static Lagu_2511531018 head_1018 = null;
    static Lagu_2511531018 tail_1018 = null;
    // fungsi menambah lagu di akhir
    public static void tambahLagu_1018(String
judul_1018, String penyanyi_1018) {
        // buat node baru
        Lagu_2511531018 laguBaru_1018 = new
Lagu_2511531018(judul_1018, penyanyi_1018);

        if (head_1018 == null) {
            head_1018 = laguBaru_1018;
            tail_1018 = laguBaru_1018;
        } else {
            // hubungkan pointer next dan prev
            tail_1018.next_1018 = laguBaru_1018;
            laguBaru_1018.prev_1018 = tail_1018;
            tail_1018 = laguBaru_1018;
        }
        System.out.println("Lagu berhasil
ditambahkan!");
    }

    // fungsi menghapus lagu pertama
    public static void hapusLaguAwal_1018() {
        // jika playlist kosong
        if (head_1018 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
    }
}

```

```

        if (head_1018 == tail_1018) {
            head_1018 = null;
            tail_1018 = null;
        } else {
            head_1018 = head_1018.next_1018;
            head_1018.prev_1018 = null;
        }
        System.out.println("Lagu pertama berhasil
dihapus!");
    }

    // fungsi tampil playlist maju
    public static void tampilMaju_1018() {
        // jika playlist kosong
        if (head_1018 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        // mulai dari head
        Lagu_2511531018 curr_1018 = head_1018;
        System.out.println("=== Playlist Maju ===");
        while (curr_1018 != null) {
            System.out.println("Judul : " +
curr_1018.judul_1018 + " | Penyanyi : " +
curr_1018.penyanyi_1018);
            // pindah ke node berikutnya
            curr_1018 = curr_1018.next_1018;
        }

        // fungsi tampil playlist mundur
        public static void tampilMundur_1018() {
            // jika playlist kosong
            if (tail_1018 == null) {
                System.out.println("Playlist kosong!");
                return;
            }

            // mulai dari tail
            Lagu_2511531018 curr_1018 = tail_1018;
            System.out.println("=== Playlist Mundur
===");
            while (curr_1018 != null) {
                System.out.println("Judul : " +
curr_1018.judul_1018 + " | Penyanyi : " +
curr_1018.penyanyi_1018);
                // pindah ke node
                sebelumnya
                curr_1018 =
curr_1018.prev_1018;
            }

            // fungsi mencari lagu berdasarkan
            judul
            public static void cariLagu_1018(String
judul_1018) {
                // jika playlist kosong
                if (head_1018 == null) {

```



```

        System.out.println("Playlist kosong!");
        return;
    }

    Lagu_2511531018 curr_1018 =
head_1018;

    boolean ditemukan_1018 = false;
    while (curr_1018 != null) {
        // pencarian tidak case-
sensitive
        if
(curr_1018.judul_1018.equalsIgnoreCase(judul_1018)) {

            System.out.println("Lagu ditemukan!");

            System.out.println("Judul : " + curr_1018.judul_1018);

            System.out.println("Penyanyi : " +
curr_1018.penyanyi_1018);

            ditemukan_1018 =
true;

            break; }
            curr_1018 =
curr_1018.next_1018;
        }

        if (!ditemukan_1018) {
            System.out.println("Lagu
tidak ditemukan!");
        }
    }

    public static void main(String[] args)
{
    Scanner input_1018 = new
Scanner(System.in);

    int pilihan_1018;

    do {
        System.out.println("\n===
Playlist Musik NIM: 2511531018 ===");
        System.out.println("1.
Tambah Lagu");
        System.out.println("2.
Hapus Lagu Pertama");
        System.out.println("3.
Lihat Playlist (Maju)");
        System.out.println("4.
Lihat Playlist (Mundur)");
        System.out.println("5.
Cari Lagu");
        System.out.println("6.
Keluar");
        System.out.print("Pilihan:
");
    }

```

```

        pilihan_1018 =
input_1018.nextInt();
        input_1018.nextLine();

        switch (pilihan_1018) {
        case 1:

            System.out.print("Judul: ");
            String judul_1018 =
input_1018.nextLine();

            System.out.print("Penyanyi: ");
            String
penyanyi_1018 = input_1018.nextLine();

            tambahLagu_1018(judul_1018, penyanyi_1018);
            break;
        case 2:

            hapusLaguAwal_1018();
            break;

        case 3:
            tampilMaju_1018();
            break;

        case 4:

            tampilMundur_1018();
            break;

        case 5:

            System.out.print("Masukkan judul lagu: ");
            String cari_1018 =
input_1018.nextLine();

            cariLagu_1018(cari_1018);
            break;

        case 6:

            System.out.println("Program selesai.");
            break;

        default:

            System.out.println("Pilihan tidak valid!");
        }

        } while (pilihan_1018 != 6);

        input_1018.close();
    }
}

```

Gambar 2.6.2 Kode program Lagu_2511531018.

2.6.3 Output

1. Menambahkan Lagu.

```

=== Playlist Musik NIM: 2511531018 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: redred
Penyanyi: cortis
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531018 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: bye
Penyanyi: ariana
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531018 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: earrings
Penyanyi: malcolm
Lagu berhasil ditambahkan!

```

Output diatas menunjukkan bagaimana program dapat menambahkan lagu sesuai dengan apa yang user masukkan. User dapat memasukkan beberapa lagu ke dalam program.

2. Menampilkan Playlist (Maju dan Mundur).

```

=== Playlist Musik NIM: 2511531018 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
=== Playlist Maju ===
Judul : redred | Penyanyi : cortis
Judul : bye | Penyanyi : ariana
Judul : earrings | Penyanyi : malcolm
Judul : the visitor | Penyanyi : sienna
Judul : work | Penyanyi : rihanna

=== Playlist Musik NIM: 2511531018 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
=== Playlist Mundur ===
Judul : work | Penyanyi : rihanna
Judul : the visitor | Penyanyi : sienna
Judul : earrings | Penyanyi : malcolm
Judul : bye | Penyanyi : ariana
Judul : redred | Penyanyi : cortis

```

Output diatas menunjukkan bagaimana program dapat menampilkan playlist sesuai dengan lagu-lagu yang telah diinput oleh user. Untuk menampilkan playlist (maju), menampilkan lagu-

lagu yang diinput secara berurut mulai dari yang pertama kali diinput oleh user. Untuk menampilkan playlist (mundur), menampilkan lagu-lagu secara berurut mulai dari yang paling akhir diinput oleh user.

3. Mencari lagu

```

=== Playlist Musik NIM: 2511531018 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: work
Lagu ditemukan!
Judul : work
Penyanyi : rihanna

=== Playlist Musik NIM: 2511531018 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: company
Lagu tidak ditemukan!

```

Output diatas menunjukkan bagaimana program dapat mencari lagu berdasarkan judul lagu yang telah dimasukkan oleh user. Tetapi pada saat user mencari lagu yang tidak diinput, maka lagu tidak ditemukan karena tidak termasuk dalam daftar playlist.

4. Hapus lagu pertama.

```

=== Playlist Musik NIM: 2511531018 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu pertama berhasil dihapus!

=== Playlist Musik NIM: 2511531018 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
=== Playlist Maju ===
Judul : bye | Penyanyi : ariana
Judul : earrings | Penyanyi : malcolm
Judul : the visitor | Penyanyi : sienna
Judul : work | Penyanyi : rihanna

```

Output diatas menunjukkan program dapat menghapus lagu pertama yang ada pada playlist. Contohnya seperti pada gambar

diatas, dalam urutan pertama itu berjudul redred-cortis, pada saat user menghapus lagu pertama, maka lagu redred-cortis terhapus dari playlist.

5. Keluar.

```
=== Playlist Musik NIM: 2511531018 ===  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
Pilihan: 6  
Program selesai.
```

Untuk menu no 6 berfungsi sebagai berhentinya program yang dijalankan.

BAB III PENUTUP

Double Linked List merupakan struktur data yang lebih fleksibel karena setiap node memiliki pointer ke node berikutnya dan node sebelumnya. Dengan adanya dua arah tersebut, proses penelusuran dan pengelolaan data menjadi lebih mudah dilakukan. Struktur data ini juga memudahkan programmer dalam melakukan operasi seperti menambah, menghapus, dan mencari data tanpa harus selalu memulai penelusuran dari awal. Selain itu, Double Linked List membantu pengguna maupun programmer dalam membuat program yang membutuhkan navigasi maju dan mundur, seperti playlist musik, galeri foto, dan riwayat browser. Oleh karena itu, penggunaan Double Linked List dapat membuat proses pemrograman menjadi lebih efisien, terstruktur, dan mudah dikembangkan.

DAFTAR PUSTAKA

- [1] R. A. Sukamto dan M. Shalahuddin, *Rekayasa Perangkat Lunak Terstruktur dan Berorientasi Objek*. Bandung: Informatika, 2018.
- [2] Rosa dan M. Shalahuddin, *Modul Pembelajaran Struktur Data*. Bandung: Informatika, 2019.
- [3] Budi Raharjo, *Belajar Otodidak Pemrograman Java*. Bandung: Informatika, 2015.